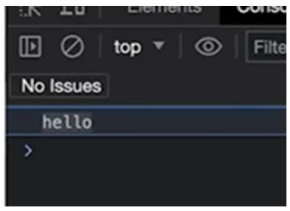
→JS-11

Call

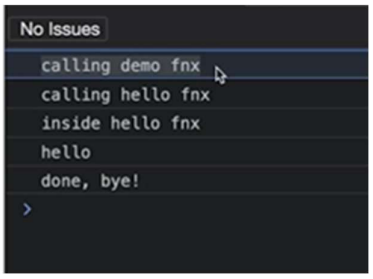
```
function hello() {
  console.log("hello");
}
function hello(): void
hello();
```

→example code:

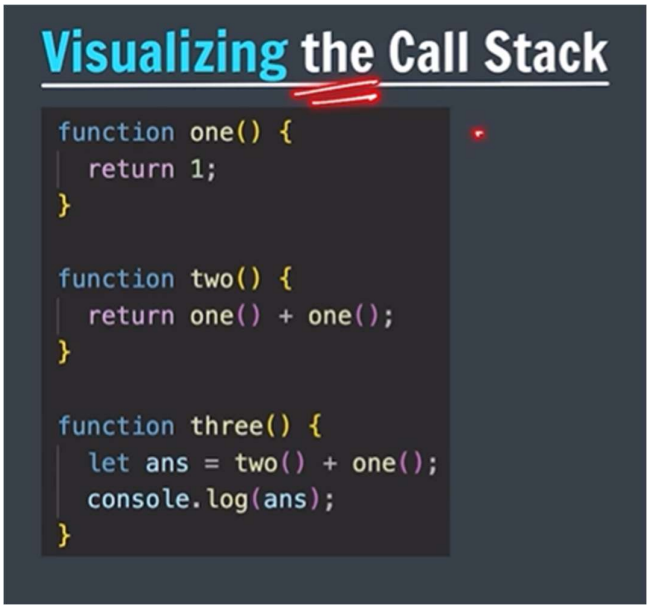
```
function hello() {
  console.log("hello");
}

function demo() {
  hello();
}

demo();
```




```
app.js > ...
function hello() {
  console.log("inside hello fnx");
  console.log("hello");
}

function demo() {
  console.log("calling hello fnx");
  hello();
}

console.log("calling demo fnx");
demo();
console.log("done, bye!");
```

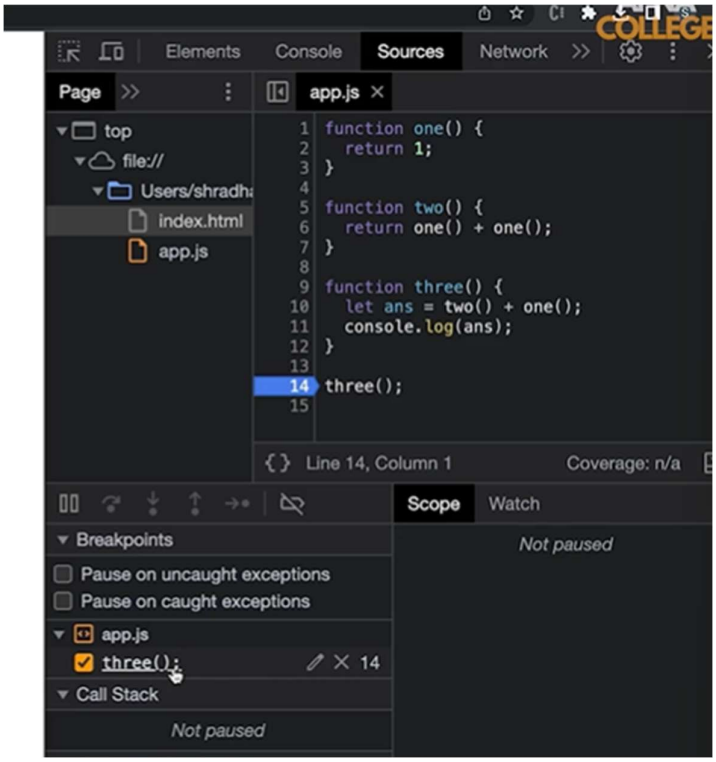


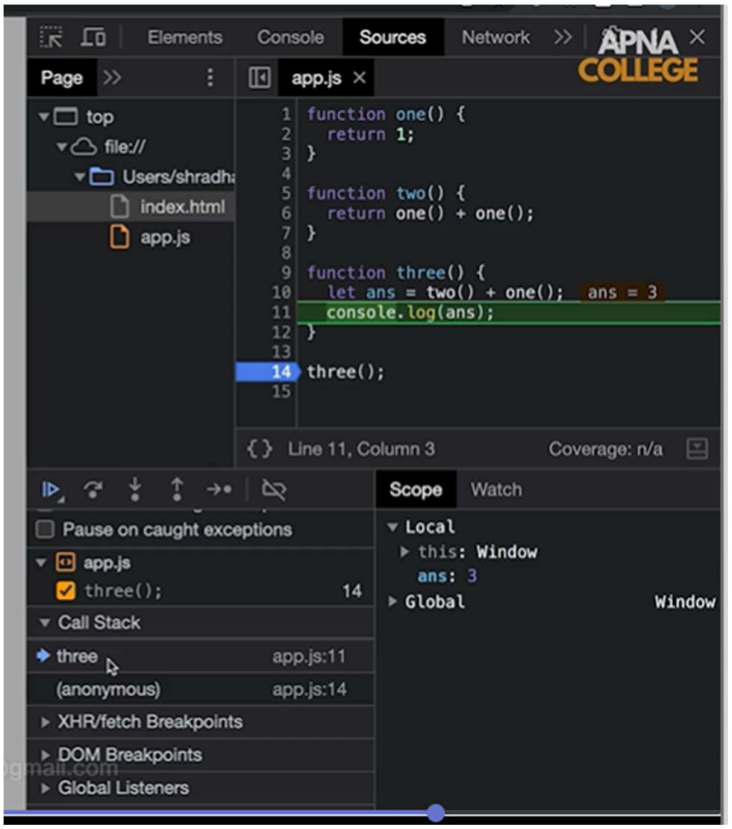
→Visualizing the call Stack



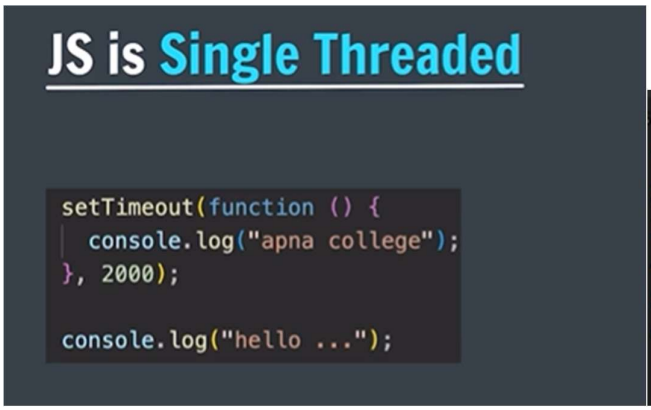
→BreakPoints:

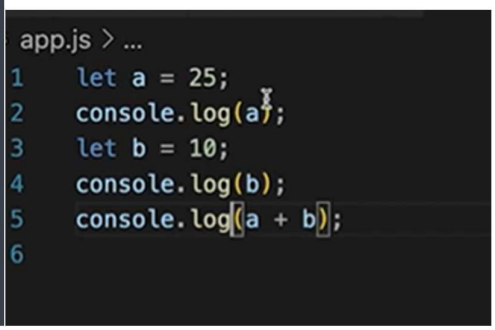
It is generally used for the debugging process

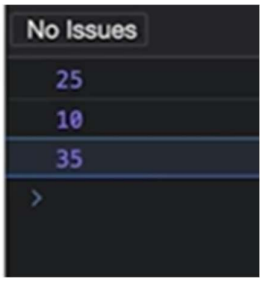


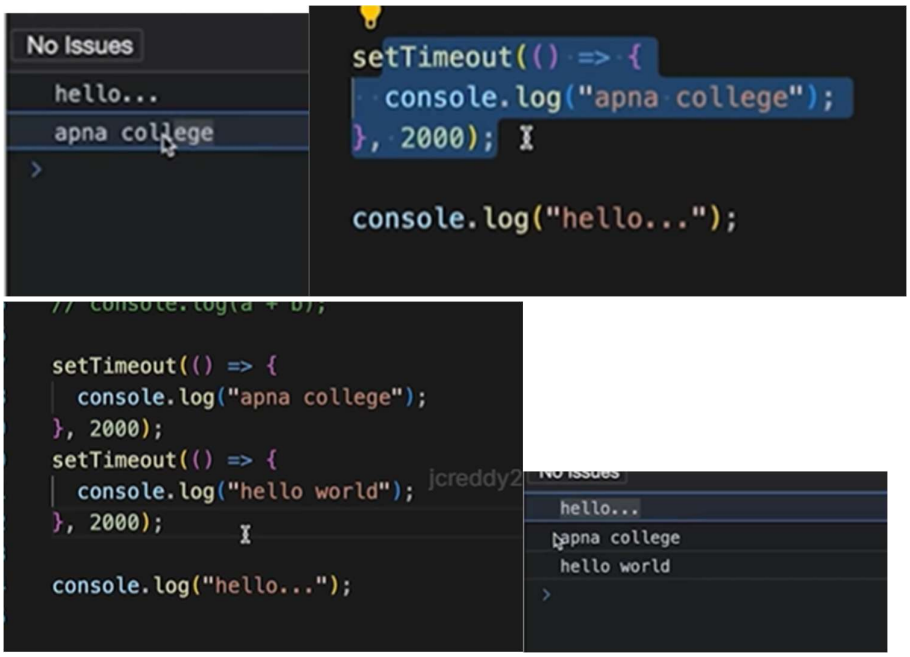


→Js is single threaded

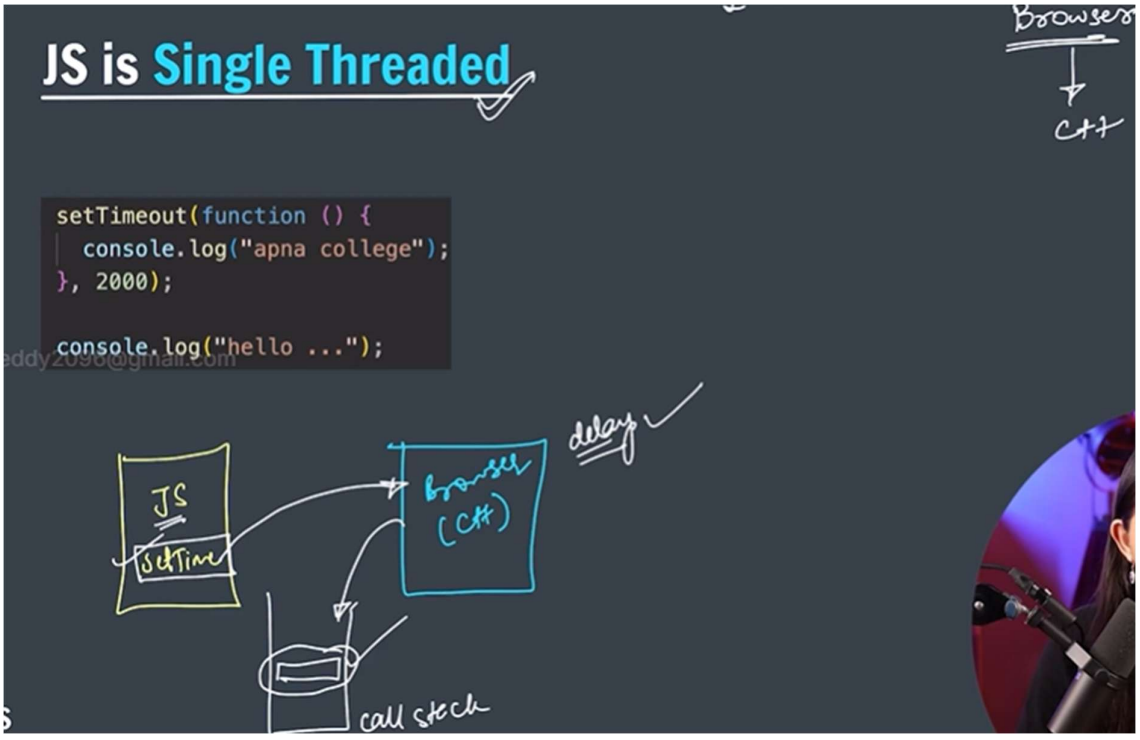








→here how it do the two works at a time when js is single threaded but this settimeout is done by the browser



→Synchronus means the normal flow of program when we use the settimeout fucntions then it is asynchronous

→Callback hell

```
h1 = document.querySelector("h1");

setTimeout(() => {
  h1.style.color = "red";
}, 1000);
```

Apna College

```
app.js > ...
h1 = document.querySelector("h1");

setTimeout(() => {
  h1.style.color = "red";
}, 1000);

setTimeout(() => {
  h1.style.color = "orange";
}, 2000);
```

Apna College

```
h1 = document.querySelector("h1");

setTimeout(() => {
  h1.style.color = "red";
}, 1000);

setTimeout(() => {
  h1.style.color = "orange";
}, 2000);

setTimeout(() => {
  h1.style.color = "green";
}, 3000);
```

Apna College

```
h1 = document.querySelector("h1");

function changeColor(color, delay) {
  setTimeout(() => {
    h1.style.color = color;
  }, delay);
}

changeColor("red", 1000);
changeColor("orange", 2000);
changeColor("green", 3000);
```

```
h1 = document.querySelector("h1");

function changeColor(color, delay, nextColorChange) {
  setTimeout(() => {
    h1.style.color = color;
    if (nextColorChange) nextColorChange();
  }, delay);
}

changeColor("red", 1000, () => {
  changeColor("orange", 1000);
});
```

app.js > changeColor("red") callback > changeColor("orange") callb

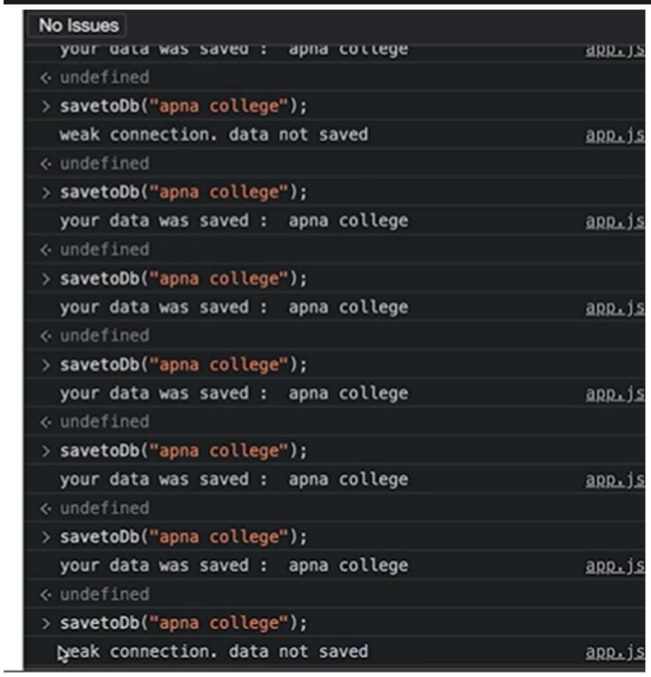
```
1   h1 = document.querySelector("h1");
2
3   function changeColor(color, delay, nextColorChange) {
4     setTimeout(() => {
5       h1.style.color = color;
6       if (nextColorChange) nextColorChange();
7     }, delay);
8   }
9
10  changeColor("red", 1000, () => {
11    changeColor("orange", 1000, () => {
12      changeColor("green", 1000, () => {
13        changeColor("yellow", 1000);
14      });
15    });
16  });
17
```

Apna College

app.js > ...

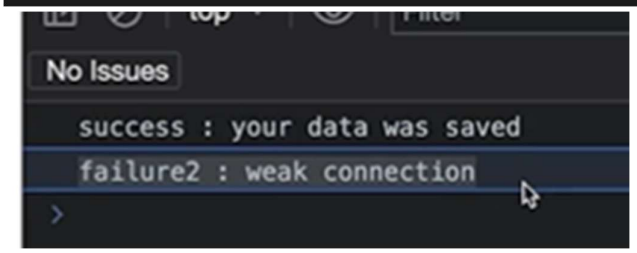
```
h1 = document.querySelector("h1");

function changeColor(color, delay, nextColorChange) {
  setTimeout(() => {
    h1.style.color = color;
    if (nextColorChange) nextColorChange();
  }, delay);
}

changeColor("red", 1000, () => {
  changeColor("orange", 1000, () => {
    changeColor("green", 1000, () => {
      changeColor("yellow", 1000, () => {
        changeColor("blue", 1000);
      });
    });
  });
});

//callbacks nesting -> callback hell
```

```
function savetoDb(data) {
  let internetSpeed = Math.floor(Math.random() * 10) + 1;
  if (internetSpeed > 4) {
    console.log("your data was saved : ", data);
  } else {
    console.log("weak connection. data not saved");
  }
}
```




```
function savetoDb(data, success, failure) {
  let internetSpeed = Math.floor(Math.random() * 10) + 1;
  if (internetSpeed > 4) {
    success();
  } else {
    failure();
  }
}

savetoDb(
  "apna college",
  () => {
    console.log("success : your data was saved");
  },
  () => {
    console.log("failure: weak connection. data not saved");
  }
);
```

```
function savetoDb(data, success, failure) {
  let internetSpeed = Math.floor(Math.random() * 10) + 1;
  if (internetSpeed > 4) {
    success();
  } else {
    failure();
  }
}

savetoDb(
  "apna college",
  () => {
    console.log("success : your data was saved");
    savetoDb(
      "hello world",
      () => {
        console.log("success2: data2 saved");
      },
      () => {
        console.log("failure2 : weak connection");
      }
    );
  },
  () => {
    console.log("failure: weak connection. data not saved");
  }
);
```



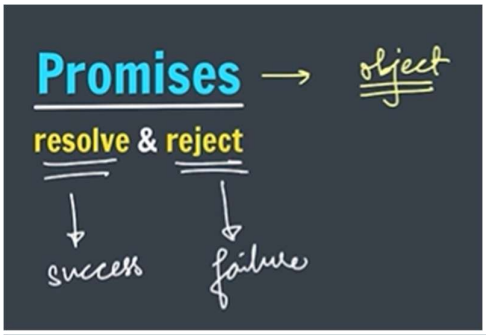

```
savetoDb(
  "apna college",
  () => {
    console.log("success : your data was saved");
    savetoDb(
      "hello world",
      () => {
        console.log("success2: data2 saved");
        savetoDb(
          "shradha",
          () => {
            console.log("success3: data3 saved");
          },
          () => {
            console.log("failure3 : weak connection");
          }
        );
      },
      () => {
        console.log("failure2 : weak connection");
      }
    );
  },
  () => {
    console.log("failure: weak connection. data not saved");
  }
);
```

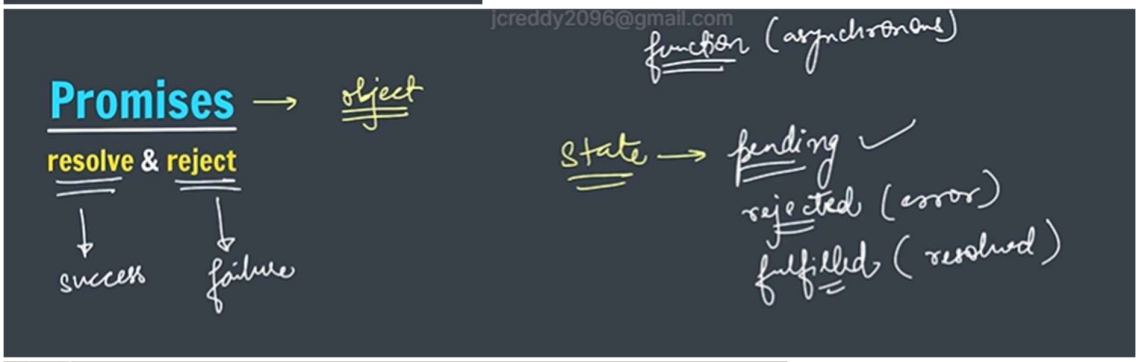
```
success : your data was saved
success2: data2 saved
failure3 : weak connection
```

→Promises

Promises

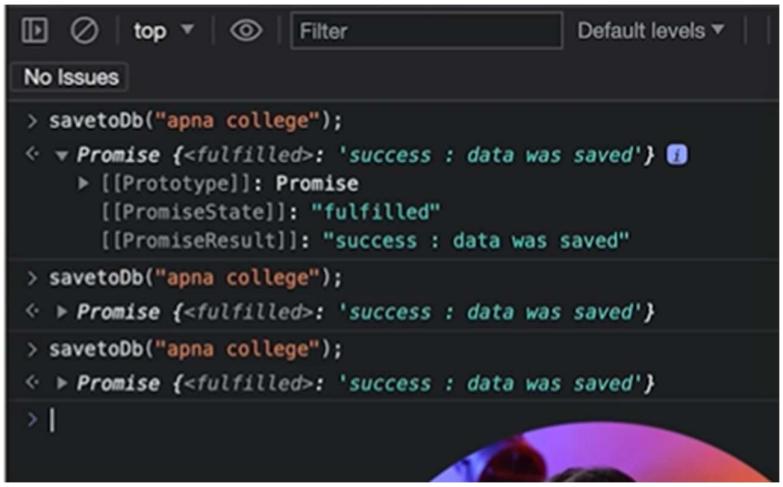
The Promise object represents the eventual completion (or failure) of an asynchronous operation and its resulting value.

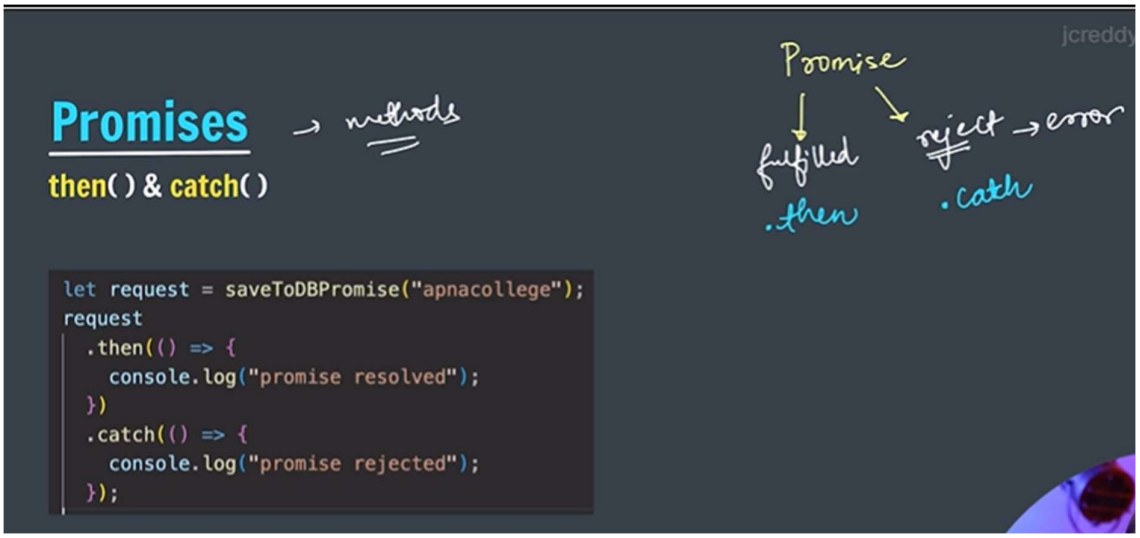





```
function savetoDb(data) {
  return new Promise((resolve, reject) => {
    let internetSpeed = Math.floor(Math.random() * 10) + 1;
    if (internetSpeed > 4) {
      success("success : data was saved");
    } else {
      failure("failure : weak connection");
    }
  });
}

// savetoDb("apna college");
```

```
function savetoDb(data) {
  return new Promise((resolve, reject) => {
    let internetSpeed = Math.floor(Math.random() * 10) + 1;
    if (internetSpeed > 4) {
      resolve("success : data was saved");
    } else {
      failure("failure : weak connection");
    }
  });
}

// savetoDb("apna college");
```





```
function savetoDb(data) {
  return new Promise((resolve, reject) => {
    let internetSpeed = Math.floor(Math.random() * 10) + 1;
    if (internetSpeed > 4) {
      resolve("success : data was saved");
    } else {
      reject("failure : weak connection");
    }
  });
}

let request = savetoDb("apna college"); //req = promise object
request.then(() => {
  console.log("promise was resolved");
})
.catch(()=>{
  console.log("promise was rejected");
})
```

```
No Issues
  promise was rejected
>
```

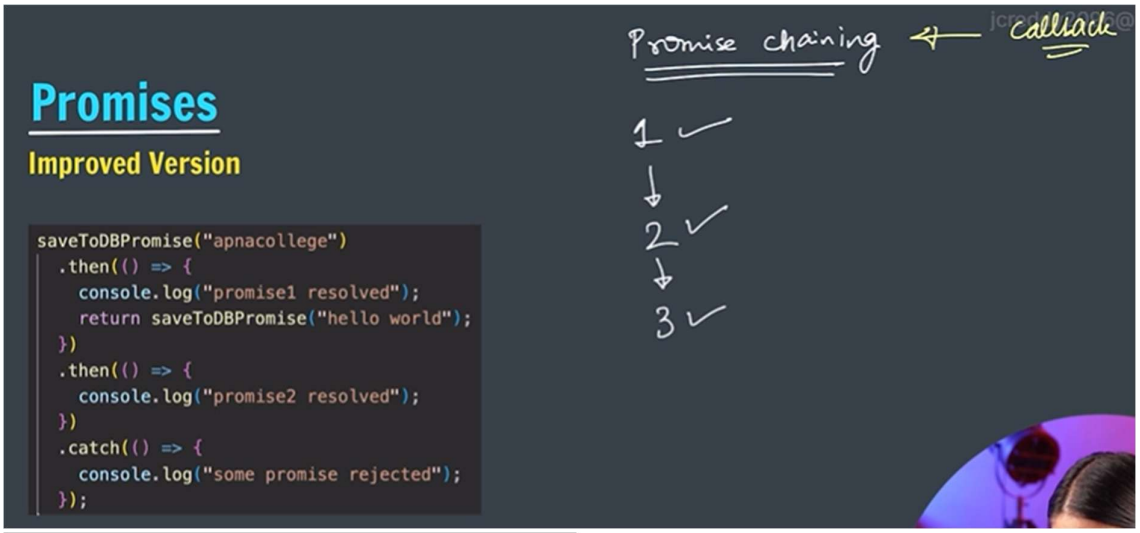
```
function savetoDb(data) {
  return new Promise((resolve, reject) => {
    let internetSpeed = Math.floor(Math.random() * 10) + 1;
    if (internetSpeed > 4) {
      resolve("success : data was saved");
    } else {
      reject("failure : weak connection");
    }
  });
}

let request = savetoDb("apna college"); //req = promise object
request
  .then(() => {
    console.log("promise was resolved");
    console.log(request);
  })
  .catch(() => {
    console.log("promise was rejected");
    console.log(request);
  });
```

```
No Issues
promise was resolved                                    app.js:71
                                                        app.js:72
▼ Promise {<fulfilled>: 'success : data was saved'}
  ▶ [[Prototype]]: Promise
    [[PromiseState]]: "fulfilled"
    [[PromiseResult]]: "success : data was saved"
>
```

```
savetoDb("apna college")
  .then(() => {
    console.log("promise was resolved");
  })
  .catch(() => {
    console.log("promise was rejected");
  });
```

```
No Issues
  promise was resolved
>
```

→Promise chaining



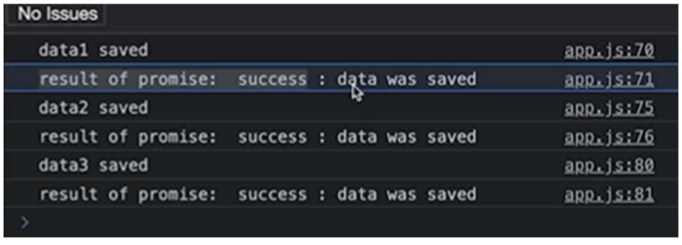

```
savetoDb("apna college")
  .then(() => {
    console.log("data1 saved");
    return savetoDb("helloworld");
  })
  .then(() => {
    console.log("data2 saved");
  })
  .catch(() => {
    console.log("promise was rejected");
  });
```

```
No Issues
data1 saved
data2 saved
>
```

```
savetoDb("apna college")
  .then(() => {
    console.log("data1 saved");
    return savetoDb("helloworld");
  })
  .then(() => {
    console.log("data2 saved");
    return savetoDb("shradha");
  })
  .then(() => {
    console.log("data3 saved");
  })
  .catch(() => {
    console.log("promise was rejected");
  });
```

```
No Issues
data1 saved                app.js:70
data2 saved                app.js:74
promise was rejected       app.js:81
>
```

→Results and errors in promises



```
h1 = document.querySelector("h1");

function changeColor(color, delay) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      h1.style.color = color;
      resolve("color changed!");
    }, delay);
  });
}

changeColor("red", 1000)
  .then(() => {
    console.log("red color was completed");
    return changeColor("orange", 1000);
  })
  .then(() => {
    console.log("orange color was completed");
    return changeColor("green", 1000);
  })
  .then(() => {
    console.log("green color was completed");
    return changeColor("blue", 1000);
  })
  .then(() => {
    console.log("blue color was completed");
  });
```